spring form #1

학습목표

- 입력 폼 구현
- 리다이렉트 과정 이해
- model attribute 전달 과정 이해
- request parameter 전달 과정 이해

목 차

1. 배경지식 2
    1) HTTP Request 2
    2) 리다이렉트 (redirect) 3
    3) 액션 메소드의 리턴값 6
    4) 절대 URL 상대 URL 6
    5) 액션 메소드의 파라미터 객체 7
2. form1 프로젝트 8
    1) 프로젝트 생성 8
    2) application.properties 10
    3) Student.java 10
    4) StudentController.java 11
    5) common.css 13
    6) student/edit.html 14
    7) student/list.html 15
3. 연습 문제 16

1. 배경지식

1) HTTP Request

웹브라우저가 웹서버에게 어떤 서비스를 요청하는 것이 HTTP Request 이다.

HTTP Request 구성 요소는 다음과 같다.

구성 요소	설명
URL	URL은 요청 대상을 가르킨다
Request Method	요청하는 작업을 나타낸다 GET: 자료 조회 요청 POST: 자료 등록 요청 PUT: 자료 수정 요청 DELETE: 자료 삭제 요청
Request Param	요청에 필요한 파라미터 값. 예를 들어 조회 요청인 경우, 조회할 학생의 id.

예:

```
  URL: http://localhost:8088/student?id=3
  Request Method: GET
  Request Param: id=3  (URL의 query string)
     설명:  id=3 student 자료 조회 요청

  URL: http://localhost:8088/student
  Request Method: POST
  Request Param: studentNo="201914099", name="홍길동"  (입력 폼에 입력된 값)
     설명:  홍길동 학생 등록 요청
```

요즘 frontend / backend 개발할 때는
GET, POST, PUT, DELETE request method를 전부 구분해서 구현해야 한다.

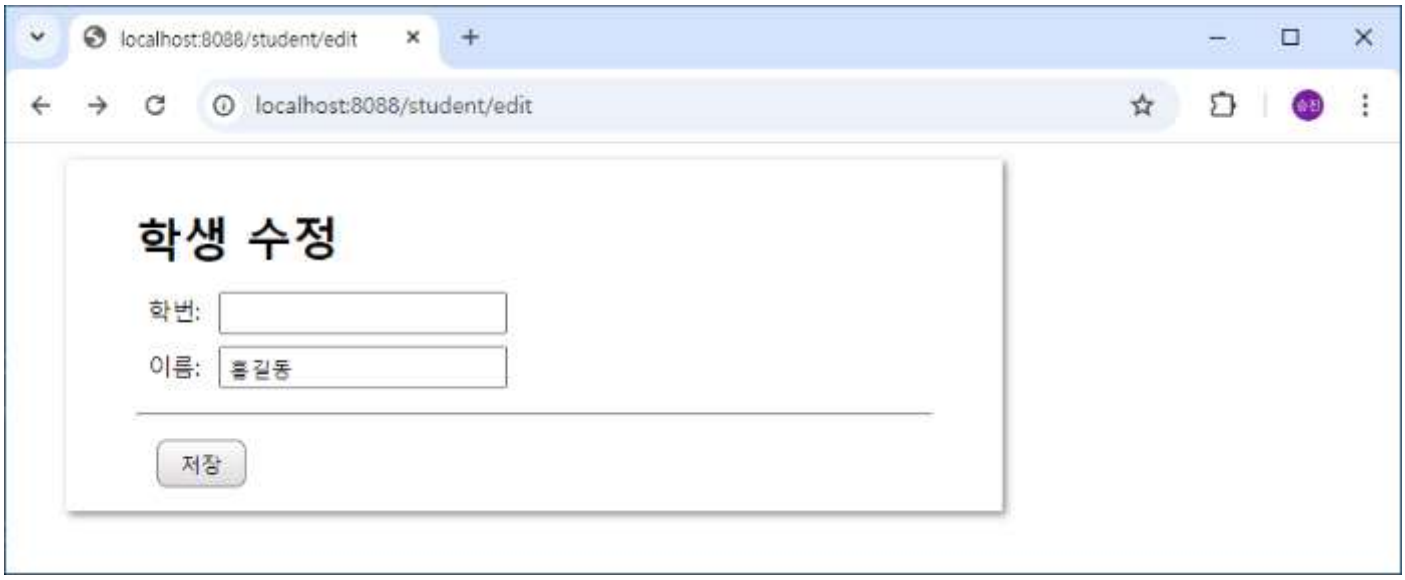
그런데, 웹 서버 프로그래밍할 때는 보통 GET, POST 두 request method만 사용해서 구현한다.
이 수업에서도 당분간 GET, POST 두 request method만 사용할 것이다.

- GET: 자료 조회 요청
- POST: 자료 등록 수정 삭제 요청

```
<form method="POST">
   이 form 내부의 submit 버튼을 클릭해서 웹서버에 HTTP request가 전송될 때는 POST 요청이다.

   위와 같이 POST, PUT, DELETE 명시하지 않은 경우에는, 기본적으로 GET 요청이 된다.
     - 직접 웹브라우저의 주소칸에 URL을 입력해서 실행할 때
     - 즐겨 찾기를 클릭했을 때
     - a 태그 링크를 클릭했을 때
```

GET, POST, PUT, DELETE 등 request method 대소문자는 중요하지 않다.



처음에 웹브라우저에서 http://localhost:8088/student/edit URL을 입력해서 실행하는 것은
GET 요청이다.

위 웹페이지의 입력 폼은 <form method="post"> 이고
저장 버튼은 <button type="submit"> 이다.
  따라서 저장 버튼을 클릭하면
  http://localhost:8088/student/edit URL이 POST 요청된다.

위 웹페이지에서 입력 태그를 요약하면 다음과 같다.

```
<form method="post">
   <input type="text" name="studentNo" />
   <input type="text" name="name" />
   <button type="submit">저장</button>
</form>
```

이 입력폼에 201914099, 홍길동 값을 입력하고
submit 버튼을 클릭하면 다음과 같은 HTTP Request가 서버에 전송된다.

URL	http://localhost:8088/student/edit
Request Method	POST
Request Param	studentNo="201914099", name="홍길동"

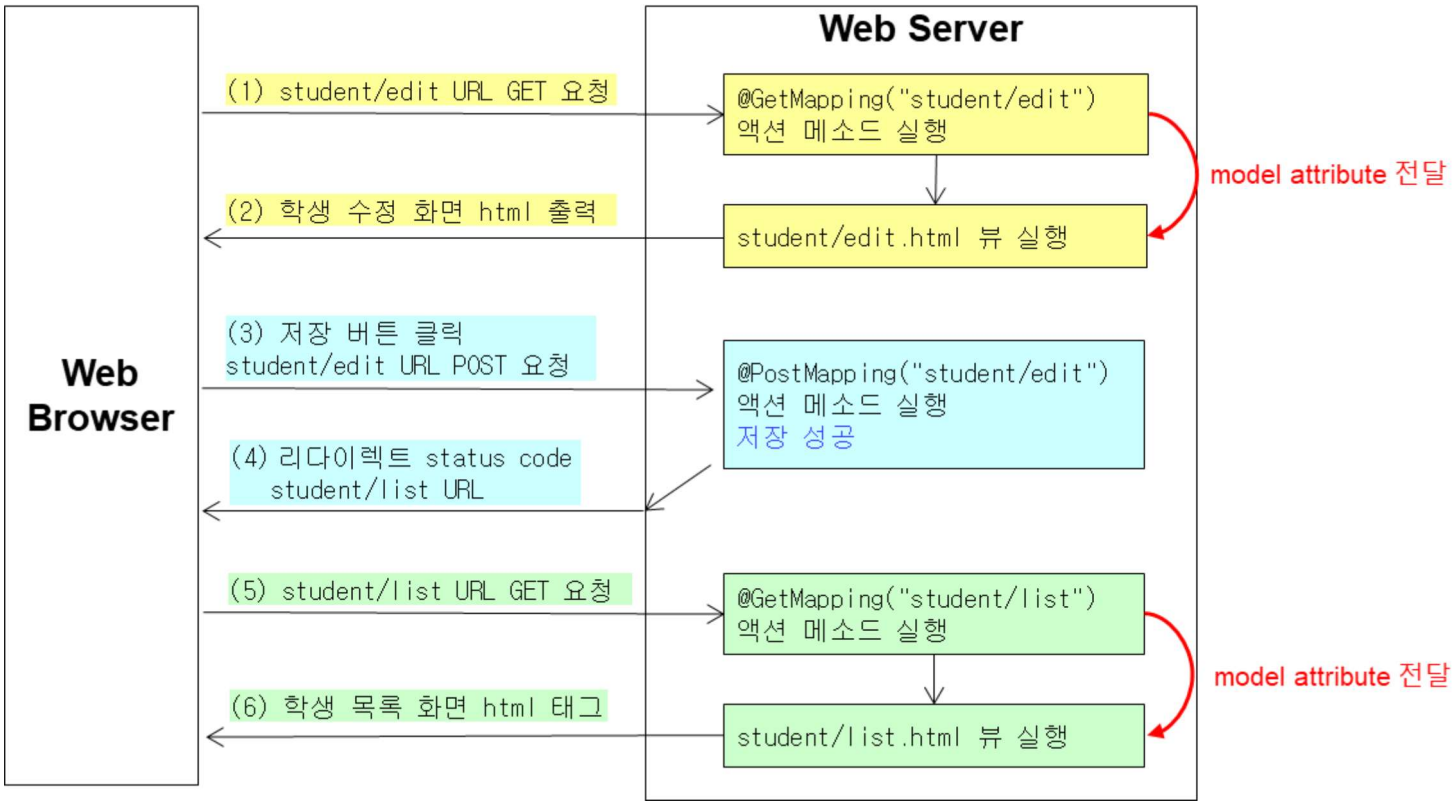
2) 리다이렉트 (redirect)

로그인 화면에서 로그인 버튼을 눌렀을 때,
   로그인에 실패하면, 에러 메시지와 함께 로그인 화면이 다시 나타난다.
   로그인에 성공하면, 메인 페이지로 넘어간다.

게시글 등록 화면에서 저장 버튼을 누르면,
   게시글 등록에 실패하면, 에러 메시지와 함께 게시글 등록 화면이 다시 나타나고,
   게시글 등록에 성공하면, 게시글 목록 페이지로 넘어간다.

이렇게 다른 페이지로 넘어가는 기능을 리다이렉트(redirect) 라고 부른다.

리다이렉트 과정 설명 - 저장 성공



(1) 웹브라우저에서 사용자가 http://localhost:8088/student/edit URL을 입력하고 엔터키를 누른다.
이것은 GET 요청이다.

서버에서 @GetMapping("student/edit") 액션 메소드가 실행된다.

(2) 서버에서 student/edit.html 뷰가 실행되고,
학생 수정 화면 html 태그가 웹브라우저에 전송된다.

(3) 웹브라우저 학생 수정 화면에서 사용자가 데이터를 입력하고 저장 버튼을 클릭한다.
http://localhost:8088/student/edit URL이 POST 요청된다.
입력된 데이터가 request parameter 데이터가 되어 서버에 전송된다.

서버에서 @PostMapping("student/edit") 액션 메소드가 실행된다.
전송된 request parameter 데이터가 액션 메소드에 전달된다.
액션 메소드에 전달된 데이터를 DB에 저장한다.

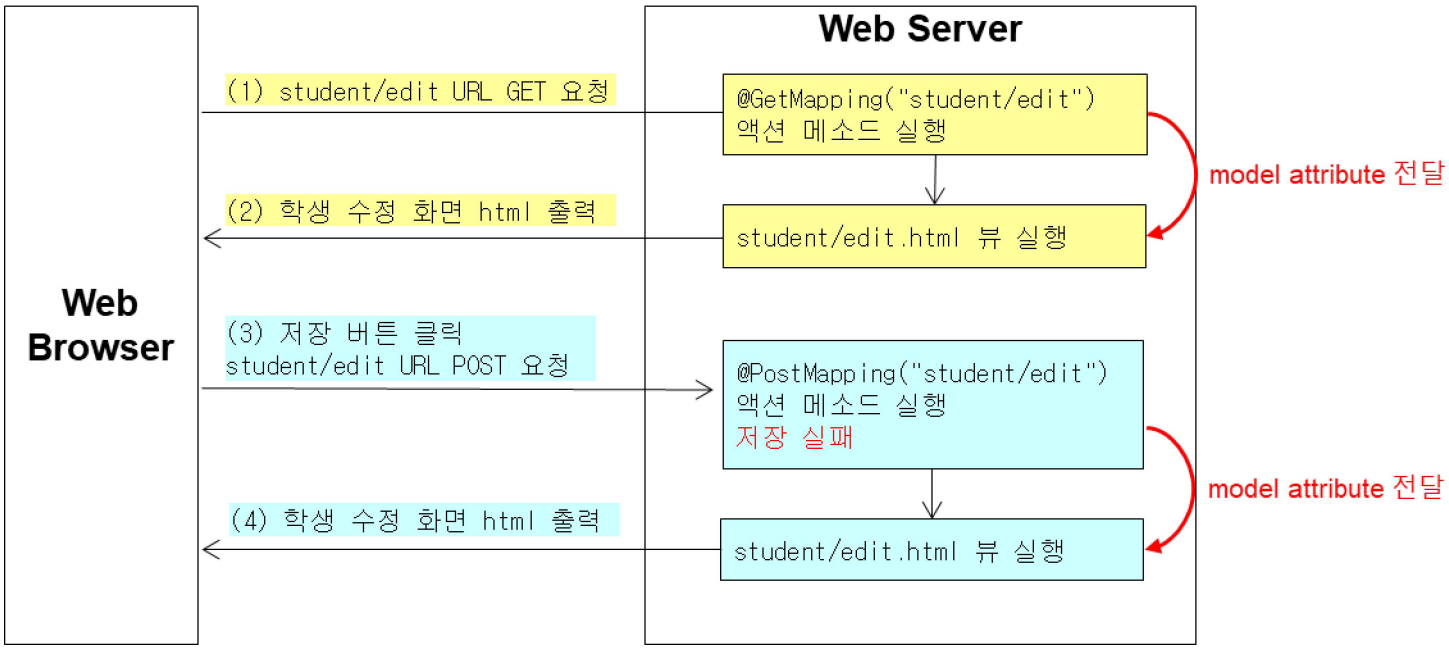
(4) 저장 성공하면, 액션 메소드는
학생 목록 http://locahost:8088/student/list URL로 리다이렉트 하라는 지시를 리턴한다.

(5) 리다이렉트하라는 지시를 서버로부터 전달 받은 웹브라우저는,
전달된 리다이렉트 URL을 즉시 서버에 다시 GET 요청한다.
즉 http://locahost:8088/student/list URL이 GET 요청된다.

서버에서 @GetMapping("student/list") 액션 메소드가 실행된다.

(6) 서버에서 student/list.html 뷰가 실행되고,
학생 목록 화면 html 태그가 웹브라우저에 전송된다.

리다이렉트 과정 설명 - 저장 실패



(1) 웹브라우저에서 사용자가 http://localhost:8088/student/edit URL을 입력하고 엔터키를 누른다.
이것은 GET 요청이다.

서버에서 @GetMapping("student/edit") 액션 메소드가 실행된다.
액션 메소드는 "student/edit" 뷰 경로를 리턴한다.

(2) 서버에서 student/edit.html 뷰가 실행되고,
학생 수정 화면 html 태그가 웹브라우저에 전송된다.

(3) 웹브라우저 학생 수정 화면에서 사용자가 데이터를 입력하고 저장 버튼을 클릭한다.
http://localhost:8088/student/edit URL이 POST 요청된다.
입력된 데이터가 request parameter 데이터가 되어 서버에 전송된다.

서버에서 @PostMapping("student/edit") 액션 메소드가 실행된다.
전송된 request parameter 데이터가 액션 메소드에 전달된다.
액션 메소드에 전달된 데이터를 DB에 저장한다.

(4) 저장 실패하면, 액션 메소드는 "student/edit" 뷰 경로를 또 리턴한다.

(5) 서버에서 student/edit.html 뷰가 다시 실행되고,
학생 수정 화면 html 태그와 에러 메시지 html 태그가 웹브라우저에 전송된다.

3) 액션 메소드의 리턴값

뷰 템플릿 파일의 경로

뷰 템플릿 파일을 실행하려면, 그 뷰 템플릿 파일의 경로를 리턴한다.
예를 들어
  student/edit.html 뷰 파일을 실행하려면, 다음과 같이 리턴한다.
  return "student/edit";

리다이렉트 지시

어떤 URL로 리다이렉트 하려면, 다음과 같은 형태의 문자열을 리턴한다.
  return "redirect:리다이렉트_URL";

리다이렉트할 URL은 상대URL 형태나 절대URL 형태로 입력한다.
  상대URL 형태: return "redirect:list";
  절대URL 형태: return "redirect:/student/list";

4) 절대 URL 상대 URL

절대 URL (absolute URL)

절대 URL은 다음과 같은 두 형태이다.

절대 URL #1 - 서버 주소까지 포함
  http://localhost:8088/student/edit

절대 URL #2 - 서버 주소만 생략
  /student/edit

즉 절대 URL은 http로 시작하거나 / 문자로 시작해야 한다.

상대 URL (relative URL)

상대 URL은 알파벳 문자로 시작한다.
현재 실행중인 액션 메소드 URL과, 다음 목적지 URL과의 차이만 적는 것이 상대 URL 이다.

예를 들어, 현재 실행중인 액션 메소드 URL이
  http://localhost:8088/student/edit 이라고 할 때,
  다음 목적지 URL이 http://localhost:8088/student/list 이라면,
  http://localhost:8088/student/ 부분까지는 같고, list 부분만 다르다. 상대 URL은 list 이다.

    상대URL 리다이렉트: return "redirect:list";
    이렇게 현재 URL과 다음 목적지 URL이 서로 끝 부분 단어 하나만 다를 때, 상대 URL을 사용한다.

  그런데 만약
    현재 URL이 http://localhost:8088/student/edit 이고
    다음 URL이 http://localhost:8088/professor/list 이라면, URL 중간 부분부터 서로 다르다.
    이런 경우에는 절대 URL을 사용한다.

    절대URL 리다이렉트: return "redirect:/professor/list";
                                  절대URL은 / 문자로 시작해야 한다.

5) 액션 메소드의 파라미터 객체

```
@PostMapping("edit")
public String edit(Model model, Student student) {
```

액션 메소드의 파라미터 `Student student` 객체

액션 메소드의 파라미터가 객체일 때, 다음과 같은 일들이 자동으로 실행된다.

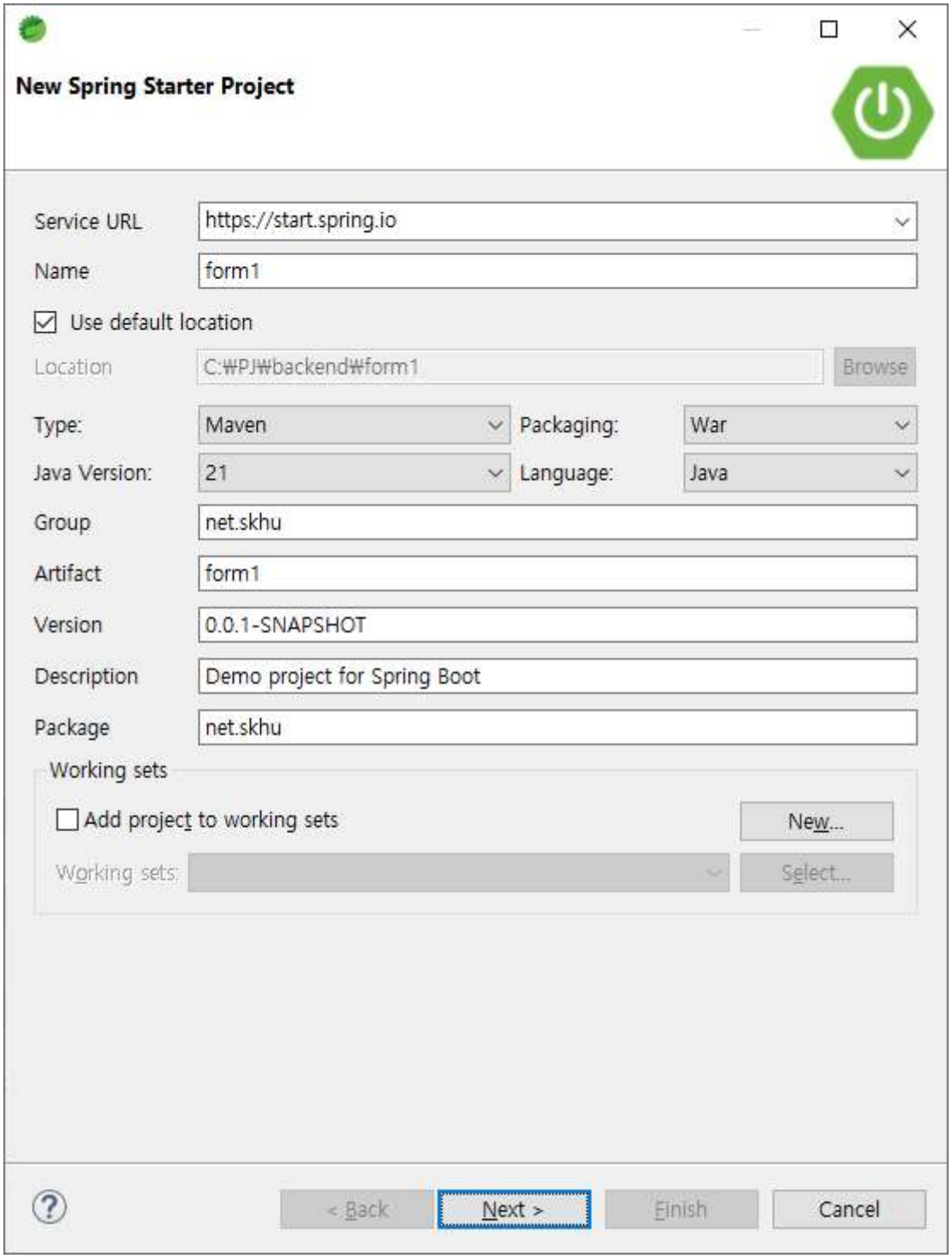
(1) request parameter 데이터가 `Student student` 객체의 채워진다.
이때 `Student` 객체의 set 메소드 이름과 request parameter 데이터의 이름이 일치하는 것만 채워진다.

예: `studentNo="201914099"` 데이터가 `student.setStudentNo("201914099")` 채워지고
`name="홍길동"` 데이터가 `student.setName("홍길동")` 채워진다.

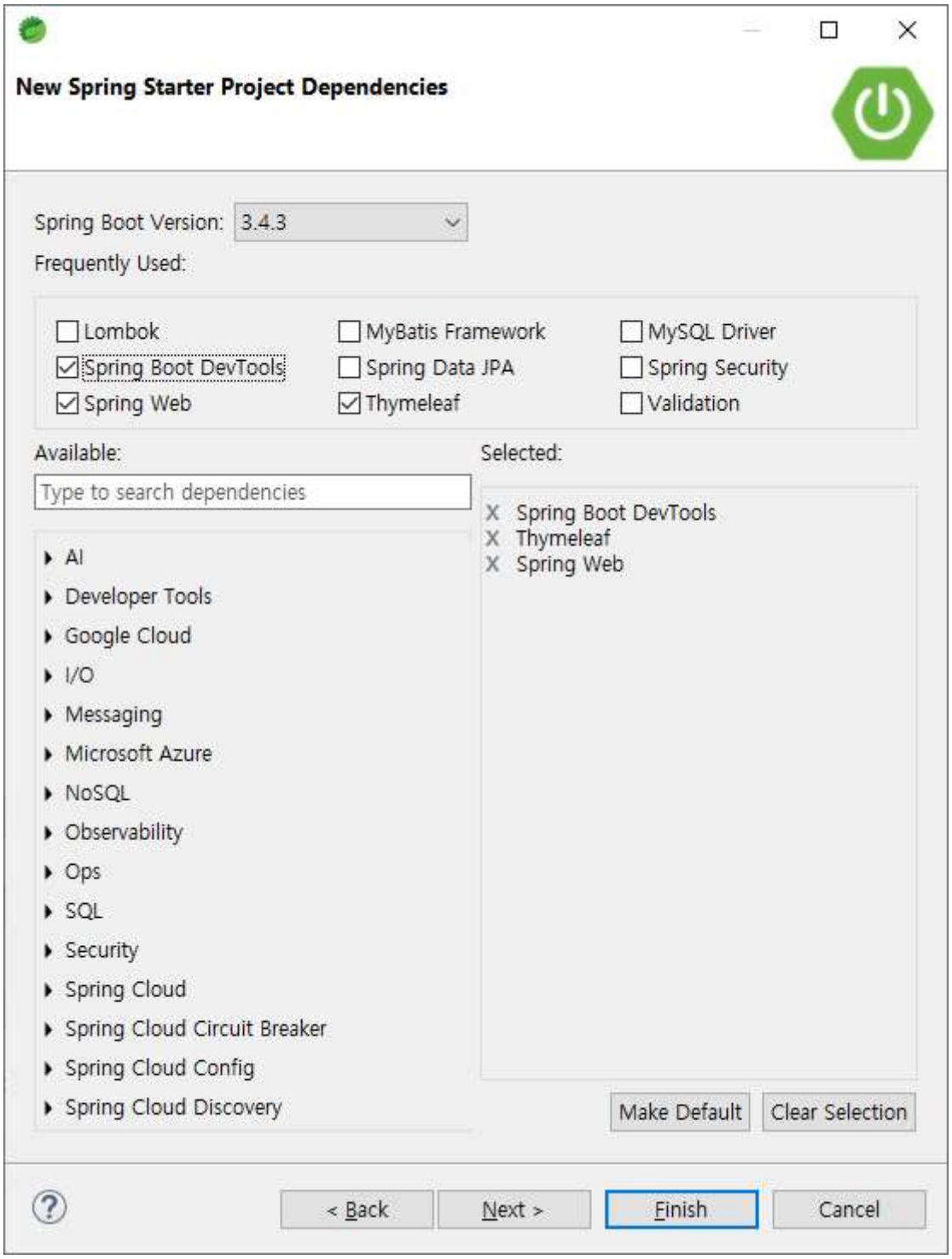
(2) `Student student` 객체가 model에 자동으로 addAttribute 된다.
이때 model attribute 이름은 `Student` 클래스 이름에서 첫 문자만 소문자로 변경한 것.
예: `model.addAttribute("student", student);`

2. form1 프로젝트

1) 프로젝트 생성



Name	form1
Type	Maven
Javer Version	21
Package	net.skhu



Spring Boot DevTools
Thymeleaf
Spring Web

2) application.properties

src/main/resources/application.properties

```
spring.application.name=form1
server.port=8088
```

3) Student.java

src/main/java/net/skhu/dto/Student.java

```java
package net.skhu.dto;

public class Student {
    String studentNo;
    String name;

    public String getStudentNo() {
        return studentNo;
    }

    public void setStudentNo(String studentNo) {
        this.studentNo = studentNo;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

학번과 이름 데이터를 채워서 전달하기 위한 DTO 클래스.

getStudentNo, setStudentNo 두 메소드에 의해서 studentNo 속성이 정의된다.
getName, setName 두 메소드에 의해서 name 속성이 정의된다.

4) StudentController.java

src/main/java/net/skhu/controller/StudentController.java

```java
package net.skhu.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.util.StringUtils;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import net.skhu.dto.Student;

@Controller
@RequestMapping("student")
public class StudentController {

    @GetMapping("edit")
    public String edit(Model model) {
        Student student = new Student();
        student.setName("홍길동");
        model.addAttribute("student", student);
        return "student/edit";
    }

    @PostMapping("edit")
    public String edit(Model model, Student student) {
        try {
            if (StringUtils.hasText(student.getStudentNo()) == false)
                throw new Exception("학번을 입력하세요");
            if (StringUtils.hasText(student.getName()) == false)
                throw new Exception("이름을 입력하세요");

            // student 객체를 DB에 저장하는 구현 생략
            System.out.println(student.getStudentNo() + " " + student.getName());
            return "redirect:list";
        }
        catch (Exception ex) {
            model.addAttribute("errorMsg", ex.getMessage());
            return "student/edit";
        }
    }

    @GetMapping("list")
    public String list() {
        // 학색 목록을 DB에서 조회해서 뷰에 전달하는 코드 생략
        return "student/list";
    }
}

```

```java
@Controller
@RequestMapping("student")
public class StudentController {

    @GetMapping("edit")
    public String edit(Model model) {

        이 액션 메소드를 호출하기 위한 URL은
        http://localhost:8088/student/edit 이다.
        이 URL이 GET 요청될 때 이 액션 메소드가 호출된다.


    @PostMapping("edit")
    public String edit(Model model, Student student) {

        이 액션 메소드를 호출하기 위한 URL은
        http://localhost:8088/student/edit 이다.
        이 URL이 POST 요청될 때 이 액션 메소드가 호출된다.
```

```java
@PostMapping("edit")
public String edit(Model model, Student student) { // 액션 메소드의 파라미터 Student student 객체
    try {
        // 학번이 입력되지 않았으면 exception을 throw 한다.
        if (StringUtils.hasText(student.getStudentNo()) == false)
            throw new Exception("학번을 입력하세요");

        // 이름이 입력되지 않았으면 exception을 throw 한다.
        if (StringUtils.hasText(student.getName()) == false)
            throw new Exception("이름을 입력하세요");

        // 학번과 이름이 모두 입력되었으면 DB에 저장하고, "student/saved" URL로 리다이렉트 한다.
        // student 객체를 DB에 저장하는 구현 생략
        System.out.println(student.getStudentNo() + " " + student.getName());
        return "redirect:list";
    }
    catch (Exception ex) {
        // try 블럭 실행 도중 exception이 throw 되면 여기로 점프하게 된다.
        model.addAttribute("errorMsg", ex.getMessage()); // 에러 메시지를 뷰에 전달한다
        return "student/edit"; // 학생 수정 화면을 다시 출력한다.
    }
}
```

StringUtils 클래스 import에 주의하자.
같은 이름의 클래스가 많기 때문에 엉뚱한 것을 import 하는 실수 하기 쉽다.

System.out.println(...) 메소드의 출력은 eclipse의 console 창에 출력된다.

5) common.css

src/main/resources/static/common.css

```css
div.container { width: 800px; margin: 10px auto; font-size: 11pt; }
.shadow { box-shadow: 2px 2px 5px gray; }

.btn { padding: 0.4em 1em; border: 1px solid gray;
       border-radius: 0.5em; background: linear-gradient(#fff, #ddd);
       text-decoration: none; color: black;
       display: inline-block; }
.btn:active {
      -ms-transform: translateY(2px);
      -webkit-transform: translateY(2px);
      transform: translateY(2px);
      background: #ccc; }

table.list { border-collapse: collapse; width: 100%; }
table.list td { padding: 4px; border: 1px solid gray; }
table.list th { padding: 4px; border: 1px solid gray;  background-color: #eee; }

div.row { margin-bottom: 0.8em; }
div.row > label:nth-child(1) { display: inline-block; min-width: 4em; text-align: right; }

input[type=text], input[type=number] { padding: 0.3em; }
select { padding: 0.3em; }
h1 { margin: 0.5em 0; }
hr { margin: 1em 0; }
.error { color: red; padding: 0.2em; margin-left: 1em; }
```

CSS 서식에 대해서는, 웹개발 입문과 웹프로그래밍에서 충분히 배웠을 테니, 설명 생략함.
모르는 부분이 있으면, 카톡으로 질문.

6) student/edit.html

src/main/resources/templates/student/edit.html

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" href="/common.css" />
  <style>
    form { width: 500px; padding: 1em 3em; }
    button { margin-left: 1em; }
  </style>
</head>
<body>
<div class="container">
 <form method="post" th:object="${student}" class="shadow">
    <h1>학생 수정</h1>
    <div class="row">
      <label>학번:</label>
      <input type="text" th:field="*{studentNo}" />
    </div>
    <div class="row">
      <label>이름:</label>
      <input type="text" th:field="*{name}" />
    </div>
    <hr />
    <button type="submit" class="btn">저장</button>
    <span class="error" th:text="${ errorMsg }"></span>
  </form>
</div>
</body>
</html>
```

${student} 부분과 ${ errorMsg } 부분은 액션 메소드로부터 전달된 model attribte 데이터이다.
   예: model.addAttribute("student", ...);
       model.addAttribute("errorMsg", ...);
        위와 같이 전달하는 코드가 StudentController 클래스의 어디에 있는지 찾아보라.

입력 폼 태그 요약

```
 <form method="post" th:object="${student}" class="shadow">
    <input type="text" th:field="*{studentNo}" />
    <input type="text" th:field="*{name}" />
    <button type="submit" class="btn">저장</button>
  </form>
```

  *{studentNo} 부분과 *{name} 부분은 th:object="${student}" 객체의 속성이다.

 위 뷰 템플릿 파일이 서버에서 실행되어, 입력 폼 html 태그들이 웹브라우저에 출력될 때,
    <input type="text" th:field="*{studentNo}" />
      이 소스 코드에 의해서
      student 객체의 studentNo 속성값이 채워진 input 태그가 출력되고,

    <input type="text" th:field="*{name}" />
      이 소스 코드에 의해서
      student 객체의 name 속성값이 채워진 input 태그가 출력된다.

위 구현은 thymeleaf 뷰 파일 소스코드이다.
위 소스 코드가 실행되면, 어떤 html 태그들이 출력되는지 확인해 보라. (필기 시험 예상 문제임)

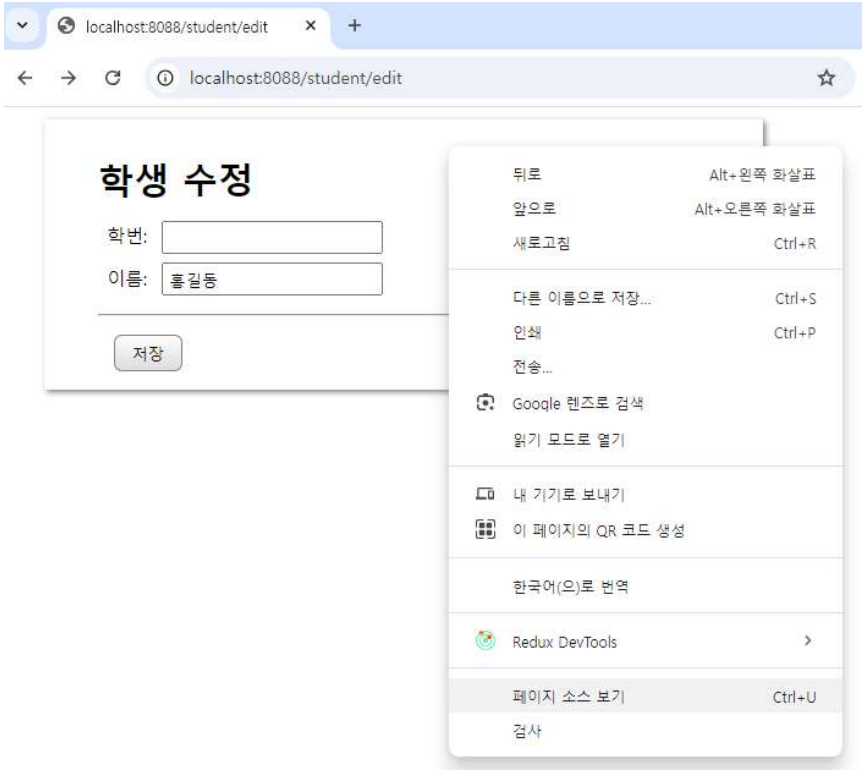
7) student/list.html

src/main/resources/templates/student/list.html

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" href="/common.css" />
  <style>
     div.container { width: 500px; padding: 2em; }
  </style>
</head>
<body>
<div class="container shadow">
   <h1>학생 목록</h1>
   <p>학생 목록 출력 생략</p>
</div>
</body>
</html>
```

3. 연습 문제

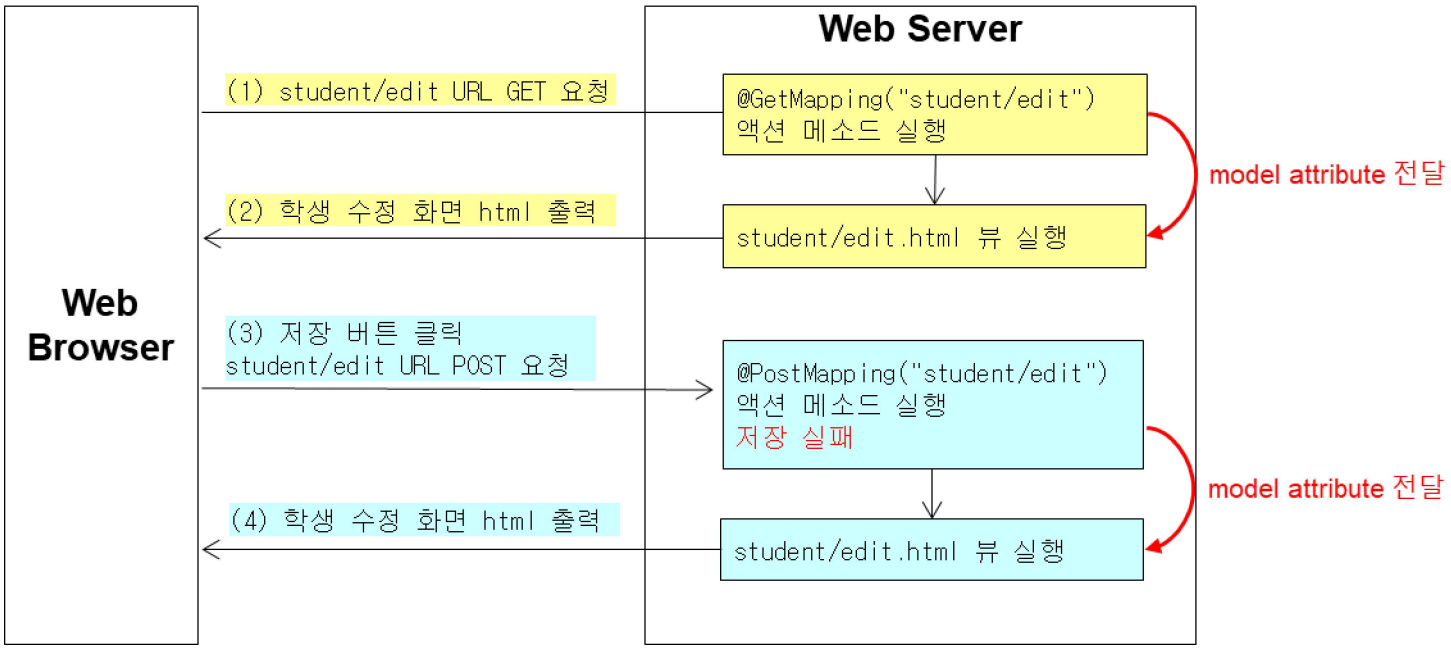
웹브라우저 창에서 소스보기



웹브라우저 창에서 '소스보기' 메뉴 클릭

`student/edit.html` 뷰 템플릿 파일이 서버에서 실행되어 웹브라우저로 출력된 `html` 태그들을 살펴보자.

리다이렉트 과정 설명 - 저장 실패



빨간색으로 표시한 두 model attribute 전달 부분에서
각각 어떤 데이터가 전달되었는가?
그 전달 과정을 구현한 소스 코드는 어느 부분인가?